Lab Assignment 8

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC

Submitted By:

Mridul Sharma

Roll No. 1024150204

Batch: 2O23

Function Template

Write a function template to swap two variables of any data type.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}
int main() {
    int x = 10, y = 20;
    swapValues(x, y);
    cout << "After swap: " << x << " " << y;
    return 0;
}
```

Output

Clear

```
After swap: 20 10

=== Code Execution Successful ===
```

Write a function template to find the minimum element in an array.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
T findMin(T arr[], int n) {
    T min = arr[0];
    for(int i = 1; i < n; i++) {
        if(arr[i] < min)
            min = arr[i];
    }
    return min;
}
int main() {
    int arr[] = {5, 2, 8, 1, 9};
    cout << "Minimum: " << findMin(arr, 5);
    return 0;
}

```

Output

Clear

```
Minimum: 1

=== Code Execution Successful ===
```

Write a function template to sort an array using bubble sort.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
void bubbleSort(T arr[], int n) {
    for(int i = 0; i < n-1; i++) {
        for(int j = 0; j < n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                T temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
int main() {
    int arr[] = {5, 3, 8, 1};
    bubbleSort(arr, 4);
    for(int i = 0; i < 4; i++)
        cout << arr[i] << " ";
    return 0;
}
```

Output

Clear

```
1 3 5 8

=== Code Execution Successful ===
```

Write a function template to perform linear search.

main.cpp

```cpp
#include <iostream>
using namespace std;
template <typename T>
int linearSearch(T arr[], int n, T key) {
    for(int i = 0; i < n; i++) {
        if(arr[i] == key)
            return i;
    }
    return -1;
}
int main() {
    int arr[] = {10, 20, 30, 40};
    int index = linearSearch(arr, 4, 30);
    cout << "Index: " << index;
    return 0;
}
```

Output

```
Index: 2

=== Code Execution Successful ===
```

Design overloaded versions of a function template process() such that:

a) Accepts a single parameter

b) Accepts two parameters of the same type

c) Accepts two parameters of different types

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
// Single parameter
template <typename T>
void process(T a) {
    cout << "Single param: " << a << endl;
}
// Two same type
template <typename T>
void process(T a, T b) {
    cout << "Same type params: " << a << " " << b << endl;
}
// Two different types
template <typename T, typename U>
void process(T a, U b) {
    cout << "Different types: " << a << " " << b << endl;
}
int main() {
    process(10);
    process(10, 20);
    process(10, 2.5);
    return 0;
}
```

Output Clear

```
Single param: 10
Same type params: 10 20
Different types: 10 2.5


=== Code Execution Successful ===
```

Class Template

Develop a class template for stack with push and pop operations.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
class Stack {
    T arr[100];
    int top;
public:
    Stack() { top = -1; }
    void push(T val) {
        arr[++top] = val;
    }
    void pop() {
        if(top >= 0)
            top--;
    }
    void display() {
        for(int i = 0; i <= top; i++)
            cout << arr[i] << " ";
    }
};
int main() {
    Stack<int> s;
    s.push(10);
    s.push(20);
    s.pop();
    s.display();
}
```

Output Clear

```
10

=== Code Execution Successful ===
```

Develop a class template for queue with enqueue and dequeue operations.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
class Queue {
    T arr[100];
    int front, rear;
public:
    Queue() { front = rear = -1; }
    void enqueue(T val) {
        if(front == -1) front = 0;
        arr[++rear] = val;
    }
    void dequeue() {
        if(front <= rear)
            front++;
    }
    void display() {
        for(int i = front; i <= rear; i++)
            cout << arr[i] << " ";
    }
};
int main() {
    Queue<int> q;
    q.enqueue(10);
    q.enqueue(20);
    q.dequeue();
    q.display();
}
```

Output

Clear

```
20

=== Code Execution Successful ===
```

Create a class template to store and display a pair of values.

main.cpp

```cpp
#include <iostream>
using namespace std;
template <typename T1, typename T2>
class Pair {
    T1 a;
    T2 b;
public:
    Pair(T1 x, T2 y) {
        a = x;
        b = y;
    }
    void display() {
        cout << a << " " << b;
    }
};
int main() {
    Pair<int, float> p(10, 2.5);
    p.display();
}
```

Output

```
10 2.5

=== Code Execution Successful ===
```

3. Create a class template for basic arithmetic operations.

main.cpp

Share Run

```cpp
#include <iostream>
using namespace std;
template <typename T>
class Arithmetic {
    T a, b;
public:
    Arithmetic(T x, T y) {
        a = x;
        b = y;
    }
    void add() { cout << "Add: " << a + b << endl; }
    void sub() { cout << "Sub: " << a - b << endl; }
    void mul() { cout << "Mul: " << a * b << endl; }
    void div() { cout << "Div: " << a / b << endl; }
};
int main() {
    Arithmetic<int> obj(10, 5);
    obj.add();
    obj.sub();
    obj.mul();
    obj.div();
}
```

Output

Clear

```
Add: 15
Sub: 5
Mul: 50
Div: 2


=== Code Execution Successful ===
```

Create a class template to input and display array elements.

Run | Debug | Stop | Share | Save | { } Beautify

main.cpp

```cpp
#include <iostream>
using namespace std;
template <typename T>
class Array {
    T arr[100];
    int n;
public:
    void input(int size) {
        n = size;
        for(int i = 0; i < n; i++)
            cin >> arr[i];
    }
    void display() {
        for(int i = 0; i < n; i++)
            cout << arr[i] << " ";
    }
};
int main() {
    Array<int> obj;
    obj.input(3);
    obj.display();
}
```

input

```
1 2 3
1 2 3

...Program finished with exit code 0
Press ENTER to exit console.
```